

AyudAPI – Exposición del equipo: Nuestra arquitectura definitiva

Modelo arquitectónico elegido

Hemos definido que AyudAPI funcionará bajo un modelo cliente-servidor. El navegador del usuario (el cliente) se encarga únicamente de mostrar la interfaz y enviar peticiones, mientras que un servidor central aloja toda la lógica de negocio, la base de datos y los archivos médicos.

¿Por qué cliente-servidor para una aplicación de emergencias con potencial uso nacional?

- Escalabilidad nacional: Podemos desplegar varias copias del servidor en diferentes regiones geográficas y utilizar un balanceador de carga para distribuir las peticiones.
- Seguridad centralizada: Los datos médicos sensibles nunca viajan completos al cliente; solo se envía lo estrictamente necesario. El control de acceso se aplica en el servidor, donde es imposible de eludir.
- Mantenimiento simplificado: Si corregimos un error o añadimos una funcionalidad, solo debemos actualizar los servidores.
- Independencia del dispositivo: Cualquier móvil con cámara y conexión a internet puede usar AyudAPI.

Dentro del modelo cliente-servidor, hemos optado por organizar el servidor como un monolito full-stack usando Next.js, en lugar de microservicios. Creemos que esto nos da la simplicidad de desarrollo y la rapidez de comunicación interna que necesita una app de emergencias.

Componentes del sistema y su entorno de despliegue

Para garantizar la disponibilidad, escalabilidad y seguridad exigidas por un sistema de salud de alcance nacional, se ha definido la siguiente infraestructura de despliegue y almacenamiento para cada componente:

1. Next.js: servidor central que unifica frontend y backend

- Función: Next.js es un framework full-stack basado en React que actúa como nuestro servidor central. Es capaz de generar las páginas web que ve el cliente y también de exponer las API que procesan los datos.
- Beneficios para el sistema:
 - Rapidez extrema en emergencias: Cuando alguien escanea un código QR, el cliente pide una página al servidor Next.js. Como Next.js puede tener esa página pre-generada (estática) o renderizada al vuelo, el tiempo de respuesta es mínimo.
 - Seguridad unificada: Toda la validación de roles (médico autenticado, usuario normal, obra social) ocurre en el mismo lugar. El cliente nunca recibe código o datos que no le correspondan.
 - Experiencia consistente para los tres roles: Usuario, Médico y Obra Social comparten la misma base de código y las mismas API.

- Entorno de despliegue:
 - Plataforma: Vercel, que ofrece soporte nativo para Next.js con CDN global y Serverless Functions.
 - Integración: Despliegue continuo desde GitHub. Cada push a la rama principal genera un despliegue automático.
 - Escalabilidad: Vercel escala horizontalmente según la demanda, ideal para picos de tráfico durante emergencias.
 - Desarrollo local: next dev en entorno local.

2. Supabase: base de datos y autenticación

Hemos elegido Supabase para gestionar la base de datos y la autenticación, por ser una solución integral que reduce la complejidad de integraciones.

2.1. PostgreSQL con PostGIS: base de datos confiable

- Función: Motor de base de datos relacional que guarda toda la información estructurada, con la extensión PostGIS preinstalada para manejar coordenadas y zonas geográficas.
- Beneficios para el sistema:
 - Integridad ACID: Las transacciones se completan por completo o no se realizan.
 - Análisis geoespacial: Permite responder preguntas como *"¿en qué provincias ha aumentado un 30% los escaneos por problemas respiratorios en la última semana?"*.
 - Flexibilidad: Columnas JSONB para datos semiestructurados (ej. lista de alergias).
- Entorno de despliegue:
 - Plataforma: Gestionada por Supabase, alojada en AWS (región us-east-1 o sa-east-1).
 - Backups: Diarios automáticos y Point-in-Time Recovery (PITR).
 - Almacenamiento: Discos SSD con replicación para alta disponibilidad.

2.2. Autenticación integrada (Auth)

- Función: Supabase Auth maneja el registro e inicio de sesión de usuarios.
- Beneficios:
 - Soporte para email/contraseña y proveedores sociales (Google, GitHub).
 - Gestión de roles mediante Row Level Security (RLS) en PostgreSQL.
 - Tokens JWT firmados, validables por Next.js.
- Entorno: Gestionado completamente por Supabase, sin infraestructura adicional.

2.3. Suscripciones en tiempo real (Realtime)

- Función: Permite que la aplicación se actualice al instante cuando hay cambios en la base de datos, usando WebSockets.
- Beneficios: Notificaciones en tiempo real, dashboards en vivo, actualizaciones de estado de emergencias.

3. MinIO: almacenamiento externo de archivos médicos

- Función: MinIO es un servicio de almacenamiento de objetos, compatible con Amazon S3, que corre en nuestros servidores. No es una base de datos, sino un *disco duro de alta velocidad* especializado en archivos grandes (radiografías, estudios, imágenes).
- Beneficios para el sistema:
 - Evita que PostgreSQL se ralentice: En la base de datos guardamos solo la ruta del archivo, y el archivo real vive en MinIO. Así la base de datos sigue siendo rápida para lo importante: perfiles, escaneos, usuarios.
 - Seguridad extrema con URLs firmadas: Cuando un médico quiere ver un estudio, el servidor Next.js le pide a MinIO una URL temporal (válida, por ejemplo, 5 minutos). Si alguien intercepta ese enlace, caducará pronto.
 - Privacidad total: Los archivos médicos nunca salen de nuestra infraestructura. No dependemos de servicios en la nube de terceros (Google Drive, Dropbox, etc.), que podrían tener acceso no autorizado. Nosotros controlamos el cifrado y las políticas de acceso, cumpliendo estrictamente con la Ley 25.326.
 - Rendimiento con alta concurrencia: MinIO está diseñado para servir cientos de archivos por segundo. En un escenario nacional con muchos médicos consultando estudios al mismo tiempo, no se convierte en un cuello de botella.
- Entorno de despliegue y almacenamiento:
 - Despliegue: Se implementará un clúster de MinIO autogestionado sobre instancias AWS EC2 dedicadas (o en un proveedor como DigitalOcean o Linode).
 - Almacenamiento: Discos SSD con cifrado AES-256 en reposo.
 - Acceso: Los archivos se entregan mediante URLs prefirmadas generadas por el backend Next.js, que valida que el usuario tenga permisos antes de generar la URL.
 - Backups: Políticas de backup diario a un bucket secundario o a AWS S3 Glacier para archivo a largo plazo.

4. Prometheus y Grafana: monitorización activa y dashboards para la obra social

- Función: Prometheus recolecta métricas del sistema; Grafana es una herramienta de visualización que crea dashboards.
- Beneficios para el sistema:
 - Transparencia para la obra social: Grafana permite construir paneles visuales y en tiempo real: enfermedades más comunes por zona, evolución de los escaneos, horas de mayor actividad.
 - Alertas proactivas: Prometheus vigila que el servidor responda en menos de 500 ms y envía alertas al equipo técnico.
 - Seguridad monitorizable: Métricas como *"número de intentos fallidos de acceso a datos médicos"* o *"escaneos desde IPs sospechosas"*.
- Entorno de despliegue y almacenamiento:
 - Despliegue: Instancia AWS EC2 independiente (tipo t3.medium) o plataforma como Render/Railway.
 - Almacenamiento de métricas: Prometheus guarda las series temporales en disco SSD con retención de 30 días.

5. OpenCode: agente de IA para acelerar el desarrollo

- Función: OpenCode es un agente de codificación de IA de código abierto que trabaja dentro de la terminal.
- Beneficios:
 - Generación de código repetitivo, refactorización, depuración asistida, análisis de datos para dashboards.
 - Gratuito, modelo-agnóstico y código abierto (licencia MIT).
- Entorno: Corre como servidor local en la máquina de cada desarrollador. No requiere almacenamiento en la nube.

6. Red de Distribución de Contenido (CDN)

- Función: Acelerar la carga de la interfaz pública de emergencia y los archivos estáticos.
- Entorno de despliegue: Vercel Edge Network (CDN global integrada) para los archivos estáticos de la aplicación. MinIO también puede configurarse con su propia CDN o cachear archivos en CloudFront.

Conclusión

Nuestra arquitectura se basa en el **modelo cliente-servidor** porque:

- Permite escalar a nivel nacional mediante réplicas del servidor y una CDN.
- Centraliza la seguridad y la lógica de negocio, protegiendo los datos médicos.
- Simplifica el mantenimiento y la actualización de la aplicación.

Dentro del servidor, hemos elegido un **monolito full-stack con Next.js** por su rapidez, su seguridad unificada y su simplicidad operativa. Acompañamos ese servidor con:

- **PostgreSQL + PostGIS** para integridad de datos y análisis geoespacial (esencial para la obra social).
- **MinIO** para almacenar archivos pesados sin ralentizar la base de datos.
- **Prometheus + Grafana** para monitorización activa y dashboards de valor.
- **Vercel, Supabase y MinIO** como plataformas de alojamiento del proyecto
- **OpenCode** como nuestro agente de IA para acelerar el desarrollo, generar código automáticamente y liberar al equipo de tareas repetitivas.

Cada herramienta fue elegida pensando primero en **cómo beneficia al usuario final** (velocidad en emergencias, seguridad de sus datos, información útil para médicos y obras sociales) y en la **viabilidad a escala nacional** (réplicas, CDN, alertas).

Además, hemos dado un paso adelante al incorporar OpenCode no solo como una característica del producto final, sino como una **herramienta de trabajo interna** que multiplica nuestra productividad como equipo.